

基于你提出的“**演进式辅助诊断**”的核心理念，应用平台域（Node.js/Express）不再仅仅是增删改查的业务后端，而是转变为一个“**事件驱动的临床状态编排器**”。它需要维持就诊会话状态，监听外部数据源增量，并智能调度推理域。

以下是对应用平台域的详细设计，包含用例分析和模块拆分：

一、用例分析

应用域主要服务于三类“参与者”：**医生（前端交互）、外部医疗系统（HIS/PACS/LIS）以及智能推理域**。

1. 核心用例图解（文字描述版）

- **Actor: 外部系统**
 - UC1: 推送增量临床数据 (HIS 推送病历文本更新)
 - UC2: 推送医学影像 (PACS 推送 DICOM/图片)
 - UC3: 推送化验结果 (LIS 推送病理JSON)
- **Actor: 医生**
 - UC4: 打开患者工作台 (初始化就诊会话)
 - UC5: 查看演进式推理结果 (实时消费 SSE 流)
 - UC6: 主动触发重试/中止 (断连感知与干预)
- **Actor: 智能推理域**
 - UC7: 接收多模态推理请求 (支持模态缺失)
 - UC8: 推送步进式流式事件 (image_done, clinical_done, report_chunk 等)
- **系统内部**
 - UC9: 异构数据归一化 (FHIR R4 解析与映射)
 - UC10: 患者主索引 (EMPI) 合并
 - UC11: 演进式推理编排 (防抖合并、增量请求、丢弃过期流)

2. 关键用例详述：演进式推理编排 (UC11)

- **触发条件：**外部系统推送了新数据，或医生补传了数据。
- **前置条件：**应用域已建立该患者的就诊会话上下文。
- **主流程：**
 - 接收外部数据，归一化后更新会话上下文。
 - 触发防抖机制，收集时间窗口内（如 1.5 秒）的所有增量数据。
 - 组装当前**全量可用模态**（病历+可能的图像+可能的病理）。
 - 若存在正在进行的旧推理流，向推理域发送取消信号或在前端丢弃旧流。
 - 携带最新全量模态向推理域发起 `POST /upload` 请求，获取 `task_id`。
 - 建立/复用与推理域的 `GET /stream/{task_id}` 连接。
 - 将推理域返回的 Agent 独立事件流式透传给前端。

二、应用域模块拆分设计

为了支撑上述用例，应用域后端建议划分为以下五大核心模块：

1. 接入与路由层

- **职责：**接收前端请求与外部系统 Webhook/API 调用，统一鉴权。
- **核心组件：**
 - `ExternalWebhookRouter`：接收 HIS/PACS/LIS 数据推送接口。根据来源打上时间戳并放入消息队列或直接交给处理层。
 - `FrontendAPPRouter`：提供医生工作台所需的 RESTful API（如获取患者列表、历史报告）和 SSE 代理接口。
 - `AuthMiddleware`：校验外部系统 API Key 及医生登录 Token。

2. 数据集成与归一化模块

- **职责：**解决多源异构数据孤岛，完成标准协议转换与主索引映射。
- **核心组件：**
 - `EMPIEngine`：患者主索引映射引擎。提供 `getUnifiedPatientID(his_patient_id, id_type)` 方法，将多系统的不同 ID 映射为系统内部唯一 PatientID。
 - `FHIRAdapter`：解析外部系统发来的 FHIR R4 资源（如 `Patient`, `Observation`, `DiagnosticReport`）。如果外部系统给的是原始 JSON/文本，在此模块包装为内部统一的 FHIR-like 结构。
 - `DataSanitizer`：数据清洗与脱敏，去除外部系统可能包含的冗余标签。

3. 会话状态管理模块 - 核心！

- **职责：**维持单个医生接诊某患者时的“增量演进状态”，这是实现流式动态重写的基础。
- **核心组件：**
 - `SessionStore`：基于 Redis 或内存的会话存储。数据结构设计：

```
{
  "session_id": "doc_001_patient_123_1698765432",
  "patient_id": "P-123",
  "context": {
    "clinical": { "data": "病历文本...", "version": 2, "updated_at": "..."},
    "image": { "data": "image_url", "version": 1, "updated_at": "..."},
  },
  "pathology": null
},
"active_task_id": "task_abc", // 当前正在跑的推理任务
"is_streaming": true
}
```

- `ContextMerger`：当新数据到达时，将其合并入 `context`，并递增 `version`。

4. 演进式推理编排模块 - 核心！

- **职责：**决定何时调用推理域，处理并发冲突，管理流式连接生命周期。
- **核心组件：**
 - `DebounceManager`：**防抖合并器**。外部系统可能瞬间推送多条字段（如病历主诉和现病史分两次发），防抖器延迟 1-2 秒，收集增量后一次性触发推理，避免频繁发起 AI 请求。
 - `InferenceDispatcher`：推理调度器。读取 `SessionStore` 的最新 context，按照推理域的 API 契约组装请求体（支持模态缺失，如 `{clinical: "...", image: null}`），发起推理请求。

- `TaskConflictResolver`: **任务冲突解决器**。如果上一个推理还在流式输出, 新数据又到了, 此组件负责:
 1. 记录新的 `task_id`。
 2. 向前端发送一个 `interrupt` SSE 事件 (让前端清空打字机, 准备重写)。
 3. 尽力向推理域发送 `cancel` 信号 (如果推理域支持断连终止), 或直接在前端逻辑中丢弃旧 Task 的后续事件。

5. SSE 代理与资源回收模块

- **职责**: 作为中继, 将推理域的流式事件透传给前端, 并双向监测连接状态。
- **核心组件**:
 - `SSEProxy`: 将从推理域接收的 `event: image_done` 等事件, 原样或封装后推给前端。
 - `ConnectionMonitor`: **断连感知器**。利用心跳机制检测前端连接状态。如果前端关闭了浏览器或离开页面, 触发 `onClose` 事件。
 - `ResourceReclaimer`: 资源回收器。接收到前端断连信号后, 立即中断与推理域的 SSE 连接, 并更新 `SessionStore.is_streaming = false`, 防止云端 4C4G 资源被无效推理占用。

三、演进式流式交互时序 (模块联动展示)

以“先来病历, 5秒后来图像”为例, 展示应用域的内部流转:

1. **T0**: HIS 推送病历文本。
 - `ExternalWebhookRouter` 接收 -> `EMPIEngine` 映射 ID -> `FHIRAdapter` 归一化。
 - `ContextMerger` 更新 `SessionStore.context.clinical`。
 - `DebounceManager` 启动 1.5s 定时器。
2. **T1.5**: 定时器触发。
 - `InferenceDispatcher` 发起推理请求 `POST /upload {clinical: "xxx", image: null, pathology: null}`。
 - 获得 `task_001`, 建立 SSE 代理。前端开始流式输出“基于病历的初步建议”。
3. **T5.0**: PACS 推送图像 DICOM。
 - `ExternalWebhookRouter` 接收 -> 归一化转码为 PNG URL。
 - `ContextMerger` 更新 `SessionStore.context.image`。
 - `DebounceManager` 再次启动 1.5s 定时器。
4. **T6.5**: 定时器触发。
 - `TaskConflictResolver` 发现 `task_001` 还在跑。
 - 向前端发送 `event: interrupt` (前端清空旧报告区, 显示“检测到新图像, 正在重新评估...”)。
 - `InferenceDispatcher` 组装最新全量数据 `POST /upload {clinical: "xxx", image: "url", pathology: null}`。
 - 获得 `task_002`, 建立新的 SSE 代理。前端开始接收 `image_done` 事件, 并流式输出新的综合报告。

四、设计优势总结

1. **屏蔽异构复杂性**: 外部系统可以是任何格式, `Data Integration` 模块将其统一, 对推理域透明。
2. **防抖与冲突兜底**: 避免了医生快速录入或外部系统批量推送导致的“请求风暴”, `TaskConflictResolver` 保证了前端始终展示的是基于最新数据的推理结果。

3. **完美的资源保护：** `ConnectionMonitor` + `ResourceReclaimer` 的存在，使得在 4C4G 极度受限的云端环境下，不会有任何“孤儿推理任务”吃满 CPU。
4. **契合临床真实节奏：** 彻底解放医生双手，实现了“数据到齐即推理，推理结果实时演进”的丝滑体验。